Lerneinheit 3 · Python-Grundlagen 3: Dateien, Fehler und Module

M1 · Python-Grundlagen

Version 1.0

Lernziele

Nach diesem Teil können Sie …

- ☐ Text- und JSON-Dateien mit `with open` lesen und schreiben,
- ☐ erklären, warum der `with`-Block eine Datei auch nach einem Fehler schließt,
- ☐ einen Traceback über mehrere Aufrufe lesen und die Zeile des eigenen Codes finden, die den Fehler verursacht,
- ☐ Fehler mit `try` und `except` gezielt abfangen,
- ☐ mit `raise` eine Ausnahme mit verständlicher Meldung auslösen,
- ☐ eigene Funktionen in ein Modul auslagern und importieren.

Teil 1 und Teil 2 des Grundkurses haben Schleifen, Funktionen und die Container behandelt. Dieser Teil behandelt drei Dinge, die jedes Programm mit Daten braucht: Dateien lesen und schreiben, Fehler behandeln und eigene Funktionen in ein Modul auslagern. Im Termin zu LE03 hat `check_input` mit `raise` einen `ValueError` ausgelöst und auf diesen Teil verwiesen; was dabei geschieht, erklärt das Kapitel „Fehler und Exceptions“. Teil 4 des Grundkurses behandelt danach Klassen, Signaturen und Stil.

Planen Sie rund anderthalb Stunden ein. Der rote Faden ist eine Ladefunktion `load_passages(path)`, die die Kursdaten aus einer JSON-Datei liest und bei einer fehlenden oder kaputten Datei eine verständliche Meldung liefert; welche Fehler sie in welchem Fall meldet, legt ein Fehlervertrag fest. Am Ende liegt sie zusammen mit `clean_words` aus Teil 2 in einem eigenen Modul `tools.py`, das jedes Notebook mit `import tools` verwenden kann. Die Projektaufgabe der nächsten Lerneinheit baut auf beidem auf, Teil 4 verwendet `tools.py` ebenfalls.

Beim Bearbeiten entstehen neben Ihrem Notebook ein Ordner `out/` für Dateien, die Sie selbst schreiben, und die Datei `tools.py` mit Ihrem Modul. Die Kursdaten in `data/` werden nie verändert. Boxen mit dem Titel „Vertiefung“ beantworten Fragen für alle, die es genau wissen wollen, und sind nicht prüfungsrelevant.

Inhalt

- 3.1 Dateien und JSON
- 3.2 Fehler und Exceptions
- 3.3 Module und Imports
- 3.4 Selbsttest

3.1 Dateien und JSON

Zuerst die Daten. Es sind dieselben wie im Termin zu LE03: die 400 Absätze und die acht Beispielanfragen als JSON, dazu die zugehörigen Vektoren als `.npy`-Dateien. Die nächste Zelle legt sie in den Ordner `data/`, falls sie dort fehlen.

```
# Vorgabe: Kursdaten bereitstellen — einfach ausführen
# Die Zelle legt den Ordner data/ an und lädt fehlende Dateien aus dem Kursrepo nach:
#   data/appetizer_passages.json
#   data/appetizer_queries.json
#   data/appetizer_embeddings.npy
#   data/appetizer_query_embeddings.npy
```

In Teil 2 war die Zelle, die die Kursdaten lädt, vorgegeben. Jetzt schreiben Sie sie selbst, denn jedes Programm, das mit Daten arbeitet, beginnt mit dem Lesen und endet oft mit dem Schreiben einer Datei. Den Anfang macht die Frage, wo eine Datei liegt.

Das Arbeitsverzeichnis. Ein relativer Pfad wie `data/appetizer_passages.json` wird vom Arbeitsverzeichnis aus gelesen, das ist der Ordner, in dem der Python-Kernel des Notebooks gestartet wurde. In JupyterLab und VS Code ist das üblicherweise der Ordner des Notebooks, in Google Colab der Ordner `/content`. Das Modul `pathlib` stellt die Klasse `Path` bereit, mit der Sie Pfade zusammensetzen und prüfen können.

```
from pathlib import Path
print(Path.cwd())           # cwd: current working directory, das Arbeitsverzeichnis als Path-Objekt
```

Die Ausgabe zeigt das Arbeitsverzeichnis Ihres Kernels. Mit `Path` setzen Sie Pfade zusammen und prüfen sie:

```
① passages_path = Path("data") / "appetizer_passages.json"
  print(passages_path)
② print(passages_path.exists(), Path("data/gibt_es_nicht.json").exists())
③ Path("out").mkdir(exist_ok=True)
```

① der Operator `/` setzt einen Pfad aus Teilen zusammen, unabhängig vom Betriebssystem
② `exists()` prüft, ob es die Datei oder den Ordner gibt
③ `mkdir(exist_ok=True)` legt einen Ordner an und meldet keinen Fehler, wenn er schon da ist

```
data/appetizer_passages.json
True False
```

Der Ordner `out/` nimmt in diesem Teil alle Dateien auf, die Sie selbst schreiben. Die Kursdaten in `data/` bleiben unverändert.

Schreiben und lesen. Die nächste Zelle schreibt eine kleine Textdatei und liest sie auf drei Arten wieder ein.

```
① with open("out/hits.txt", "w", encoding="utf-8") as f:
②     f.write("0.89  Gehirn\n")
      f.write("0.72  Zeit\n")
③ with open("out/hits.txt", encoding="utf-8") as f:
      content = f.read()
  print("read():", repr(content))
④ with open("out/hits.txt", encoding="utf-8") as f:
      lines = f.readlines()
  print("readlines():", lines)
⑤ print("Schleife:")
  with open("out/hits.txt", encoding="utf-8") as f:
      for line in f:
          score, title = line.strip().split("  ")
          print("   ", title, float(score))
```

① `open(pfad, "w", encoding="utf-8")` öffnet die Datei zum Schreiben; `with … as f` bindet die geöffnete Datei an `f` und schließt sie am Ende des eingerückten Blocks
② `write` schreibt einen String; ein Zeilenumbruch `\n` muss selbst dabeistehen
③ ohne Modus öffnet `open` zum Lesen (Modus `"r"`); `read()` liefert den ganzen Inhalt als einen String
④ `readlines()` liefert eine Liste mit einer Zeile je Eintrag, der Zeilenumbruch bleibt am Ende jeder Zeile stehen
⑤ eine Schleife über die Datei liefert die Zeilen nacheinander, ohne die ganze Datei auf einmal zu laden; `strip()` entfernt den Zeilenumbruch

```
read(): '0.89  Gehirn\n0.72  Zeit\n'
readlines(): ['0.89  Gehirn\n', '0.72  Zeit\n']
Schleife:
    Gehirn 0.89
    Zeit 0.72
```

Die Schleife über die Datei ist die übliche Form für Dateien, die zeilenweise aufgebaut sind. `read()` passt, wenn der ganze Inhalt als ein String gebraucht wird, etwa für `json`-Dateien. `readlines()` passt, wenn Sie die Zeilen als Liste weiterverarbeiten wollen.

> 💭 **Kurz überlegen**
>
> Die nächste Zelle ruft `f.read()` zweimal hintereinander im selben `with`-Block auf. Was liefert der zweite Aufruf?

```
with open("out/hits.txt", encoding="utf-8") as f:
    first = f.read()
    second = f.read()
print(repr(first))
print(repr(second))
```

```
'0.89  Gehirn\n0.72  Zeit\n'
''
```

Genau verstehen: Warum with?

`open` gibt ein Dateiobjekt zurück, und eine geöffnete Datei belegt Betriebsmittel, bis sie geschlossen wird: einen Dateizugriff beim Betriebssystem und einen Puffer im Speicher. Ohne `with` müssten Sie `f.close()` selbst aufrufen, und zwar auf jedem Ausführungspfad aus dem Code heraus, auch dann, wenn zwischen `open` und `close` ein Fehler auftritt. Der `with`-Block übernimmt das automatisch: Am Ende des eingerückten Blocks wird die Datei geschlossen, auch wenn der Block mit einer Ausnahme verlassen wird. Beim Schreiben ist das wichtig, weil `write` die Daten zunächst in einem Puffer sammelt und erst das Schließen sie vollständig in die Datei bringt. Eine Datei, die nicht geschlossen wurde, kann unvollständig auf der Platte liegen.

Das Muster ist deshalb immer dasselbe: `with open(...) as f:` und darunter die Arbeit mit `f`. Nach dem Block ist `f` geschlossen, ein weiteres `f.read()` bricht mit einem `ValueError: I/O operation on closed file` ab. Die Mechanik dahinter heißt Context Manager: ein Objekt, das beim Betreten des `with`-Blocks eine Vorbereitung ausführt und beim Verlassen ein Aufräumen, bei einer Datei das Schließen. Eigene Context Manager gehören zum Material Advanced Python.

Was `with` nicht tut: Schreibvorgänge rückgängig machen. Scheitert `json.dump` mitten in der Datei, etwa an einem NumPy-Array, ist die Datei danach zwar geschlossen, aber der alte Inhalt ist durch den Modus `"w"` schon gelöscht, und ein unvollständiger Anfang kann stehen bleiben. Auch `try` und `except` setzen nichts zurück.

Die Zeichenkodierung. `encoding="utf-8"` steht in jeder `open`-Zeile dieses Kurses. Eine Datei speichert Bytes, und die Kodierung legt fest, welche Bytes welches Zeichen bedeuten. Ohne Angabe hängt die Kodierung von der Python-Version und von der Umgebung ab: vor Python 3.15 häufig von den Einstellungen des Betriebssystems, ab 3.15 ist UTF-8 die Voreinstellung. Wir geben sie an, weil die Kursdateien in UTF-8 gespeichert sind und der Code unabhängig von der Voreinstellung funktionieren soll. Mit einer falschen Kodierung erscheinen Umlaute als unlesbare Zeichenfolgen, oder das Lesen bricht mit einem `UnicodeDecodeError` ab.

💭 Kurz überlegen

Die nächste Zelle öffnet `out/hits.txt` noch einmal mit dem Modus `"w"` und schreibt eine einzige Zeile. Was steht danach in der Datei, und was gibt die Zelle aus?

```
with open("out/hits.txt", "w", encoding="utf-8") as f:
    f.write("0.65  Koran\n")
with open("out/hits.txt", encoding="utf-8") as f:
    print(f.readlines())
```

Überblick: Dateien öffnen und lesen

Die Modi von `open` und die Lesemethoden eines Dateiobjekts `f`.

Werkzeug	Tut was	Gibt zurück
`open(path, encoding="utf-8")`	öffnet zum Lesen (Modus `"r"`), die Datei muss vorhanden sein	Dateiobjekt, bei fehlender Datei `FileNotFoundError`
`open(path, "w", encoding="utf-8")`	öffnet zum Schreiben, legt die Datei an oder löscht ihren Inhalt	Dateiobjekt
`open(path, "a", encoding="utf-8")`	öffnet zum Anhängen ans Ende	Dateiobjekt
`open(path, "rb")`	öffnet eine Binärdatei zum Lesen (siehe unten)	Dateiobjekt, `read()` liefert Bytes
`f.read()`	liest den verbleibenden Inhalt, beim ersten Aufruf alles	ein String (Textmodus)
`f.readlines()`	liest die verbleibenden Zeilen, mit Zeilenumbruch am Ende	Liste von Strings
`f.readline()`	liest die nächste Zeile	ein String, `"\n"` bei einer Leerzeile, `""` am Dateiende
`for line in f:`	liefert die Zeilen nacheinander	je Durchlauf ein String
`f.write(text)`	schreibt den String, ohne Zeilenumbruch	Anzahl der geschriebenen Zeichen
`Path("out") / "x.txt"`	setzt einen Pfad zusammen	`Path`-Objekt, lässt sich wie ein String an `open` übergeben
`path.exists()`	prüft, ob es die Datei gibt	`bool`
`Path("out").mkdir(exist_ok=True)`	legt den Ordner an	`None`

JSON. Die Kursdaten liegen als JSON vor, einem Textformat für strukturierte Daten.

Genau verstehen: Das Format JSON

JSON (JavaScript Object Notation) ist Text, der Daten nach sechs Regeln aufschreibt: Ein **Objekt** ist eine Zuordnung in geschweiften Klammern, `{"id": 6, "title": "Gehirn"}`, die Schlüssel sind immer Strings. Ein **Array** ist eine Folge in eckigen Klammern, `[1, 2, 3]`. Ein **String** steht in doppelten Anführungszeichen, eine **Zahl** steht nackt (`92`, `0.89`), dazu kommen `true`, `false` und `null`. Objekte und Arrays dürfen beliebig ineinander geschachtelt sein, Leerzeichen und Zeilenumbrüche zwischen den Teilen spielen keine Rolle. Mehr gibt es nicht; genau deshalb können fast alle Sprachen und Werkzeuge JSON lesen.

Die Datei `appetizer_passages.json` ist nach diesem Muster gebaut, hier schematisch, auf den Schlüssel `passages` reduziert und mit erfundenen Beispieldaten:

```
{"passages": [{"id": 6, "title": "Gehirn", "text": "Das Gehirn ..."},
              {"id": 7, "title": "Gehirn", "text": "..."}, ...]}
```

Die Entsprechungen sind naheliegend: Objekt ist Dict, Array ist Liste, der Rest sind String, Zahl, Wahrheitswert und `None`.

Das Modul `json` übersetzt zwischen diesem Text und Python-Objekten: `json.load` liest aus einer geöffneten Datei, `json.dump` schreibt in eine. Die nächste Zelle schreibt ein kleines Dict, zeigt den Dateiinhalt und liest ihn zurück.

```
import json

record = {"title": "Gehirn", "hit": ("Gehirn", 0.89), "ids": [92, 157], 7: "sieben", "checked": True,
"note": None}
① with open("out/record.json", "w", encoding="utf-8") as f:
    json.dump(record, f, indent=2, ensure_ascii=False)

print("Dateiinhalt:")
with open("out/record.json", encoding="utf-8") as f:
    print(f.read())
② with open("out/record.json", encoding="utf-8") as f:
    restored = json.load(f)
print("Zurückgelesen:", restored)
print("Gleich dem Original?", restored == record)
```

① `json.dump(obj, f, …)` schreibt das Objekt als JSON-Text in die Datei; `indent=2` bricht den Text lesbar um, `ensure_ascii=False` lässt Umlaute als Zeichen stehen statt als `\u00e4`

② `json.load(f)` liest den JSON-Text und baut daraus Dicts, Listen, Strings, Zahlen, `True` / `False` und `None`

```
Dateiinhalt:
{
  "title": "Gehirn",
  "hit": [
    "Gehirn",
    0.89
  ],
  "ids": [
    92,
    157
  ],
  "7": "sieben",
  "checked": true,
  "note": null
}
Zurückgelesen: {'title': 'Gehirn', 'hit': ['Gehirn', 0.89], 'ids': [92, 157], '7': 'sieben',
'checked': True, 'note': None}
Gleich dem Original? False
```

Die Rückübersetzung ist nicht in jedem Fall identisch mit dem Original, deshalb ist der Vergleich am Ende `False`. JSON kennt kein Tupel, aus `("Gehirn", 0.89)` wird eine Liste. Und die Schlüssel eines JSON-Objekts sind immer Strings, aus dem Schlüssel `7` wird `"7"`. Für die Kursdaten spielt das keine Rolle, denn dort sind alle Schlüssel Strings und alle Folgen Listen. Für eigene Daten heißt es: Was aus JSON zurückkommt, sind Dicts mit String-Schlüsseln, Listen, Strings, Zahlen, Wahrheitswerte und `None`. Objekte anderer Typen, etwa ein NumPy-Array, lassen sich nicht direkt schreiben; `json.dump` bricht dann mit einem `TypeError` ab.

Eine JSON-Datei enthält genau ein Dokument. Zwei `json.dump`-Aufrufe hintereinander ergeben etwa `{"a": 1}{"b": 2}`, und `json.load` bricht damit ab (`JSONDecodeError: Extra data`). Mehrere Datensätze kommen als eine Liste in einen Aufruf. Das Format JSON Lines, eine Zeile je Datensatz, ist ein eigenes Format mit eigenen Werkzeugen.

Überblick: Python und JSON

Die Übersetzung in beide Richtungen. `json.dumps` und `json.loads` arbeiten wie `dump` und `load`, aber mit einem String statt einer Datei.

Python	JSON	Zurück nach Python
`dict`	Objekt `{"a": 1}`	`dict`, Schlüssel immer `str`
`list`, `tuple`	Array `[1, 2]`	`list`
`str`	String `"Gehirn"`	`str`
`int`, `float`	Zahl `92`, `0.89`	`int` oder `float`
`True`, `False`	`true`, `false`	`True`, `False`
`None`	`null`	`None`
NumPy-Array, Set, eigene Objekte	nicht darstellbar: `TypeError`	

Standard-JSON kennt kein `NaN` („not a number“, in NumPy etwa das Ergebnis von 0/0, siehe LE03) und kein Unendlich. Pythons `json` schreibt `float("nan")` und `float("inf")` trotzdem als `NaN` und `Infinity`, andere Programme lehnen das ab; mit `allow_nan=False` löst `json.dump` stattdessen einen `ValueError` aus.

✏️ Aufgabe 3.1 #1 5 min

Schreiben Sie die Funktion `load_passages(path)`. Sie bekommt den Pfad einer JSON-Datei im Format der Kursdaten als String, liest die Datei mit `with open` und `json.load` und gibt die Liste der Absätze zurück, die in der Datei unter dem Schlüssel `"passages"` steht. Rufen Sie die Funktion danach für `"data/appetizer_passages.json"` auf und legen Sie das Ergebnis in der Variablen `passages` ab. Die Beispielanfragen laden Sie auf dieselbe Weise aus `"data/appetizer_queries.json"`, dort heißt der Schlüssel `"queries"`; legen Sie die Liste in der Variablen `queries` ab.

Zusatzaufgabe: Schreiben Sie in der Zelle nach der Prüfzelle eine Funktion `save_titles(records, path)`, die die Titel einer Liste von Absätzen als Textdatei speichert, ein Titel je Zeile, jeder Titel nur einmal und in der Reihenfolge des ersten Vorkommens. Schreiben Sie die Titel von `passages[:10]` nach `out/titles.txt`, lesen Sie die Datei zurück und zählen Sie die Zeilen.

Hinweis: Die Ladezelle aus Teil 2 hatte genau diese Form: `with open(path, encoding="utf-8") as f:` und darunter `json.load(f)["passages"]`. Für die Anfragen genügt ein zweiter `with`-Block ohne eigene Funktion.

→ *Lösung im Anhang*

```python
def load_passages(path):
    # TODO: Ihre Lösung hier
    ...

passages = load_passages("data/appetizer_passages.json")
# TODO: Ihre Lösung hier
...
```

```
# Prüfzelle – einfach ausführen (legt eine eigene kleine Datei in out/ an)
with open("out/check_small.json", "w", encoding="utf-8") as _f:
    json.dump({"passages": [{"id": 1, "title": "Probe", "text": "Ein Satz."}]}, _f)
assert load_passages("data/appetizer_passages.json") is not None, "load_passages gibt noch nichts zurück (return vergessen?)"
assert type(passages) == list, "load_passages gibt die Liste unter dem Schlüssel passages zurück, nicht das ganze Dict"
assert load_passages("out/check_small.json") == [{"id": 1, "title": "Probe", "text": "Ein Satz."}], "die Funktion liest die Datei, deren Pfad übergeben wird"
assert len(passages) == 400, "400 Absätze"
assert passages[108]["title"] == "San Diego", "jeder Absatz ist ein Dict mit id, title und text"
assert type(queries) == list and len(queries) == 8, "queries ist die Liste der acht Beispielanfragen"
assert queries[0]["expected"] == 108, "jede Anfrage nennt die Nummer des erwarteten Absatzes"
print("Alle Prüfungen bestanden ✓")
```

```
# Zusatzaufgabe: die Titel einer Absatzliste als Textdatei speichern und zurücklesen
# TODO: Ihre Lösung hier
...
```

✏️ Aufgabe 3.1 #2 10 min

Ein typischer Fall für JSON: Ein Zwischenergebnis wird einmal berechnet und als Datei gespeichert, und beim nächsten Start liest das Programm die Datei, statt neu zu rechnen. Hier ist das Zwischenergebnis die Titelstatistik aus Teil 2.

1. Bauen Sie aus `passages` das Dict `title_counts`, das jedem Titel die Anzahl seiner Absätze zuordnet. Das ist das Zählmuster aus Teil 2 mit `get`.
2. Schreiben Sie das Dict mit `json.dump` nach `out/title_counts.json`, lesbar umgebrochen und mit Umlauten als Zeichen.
3. Lesen Sie die Datei mit `json.load` in die neue Variable `restored_counts` zurück und prüfen Sie mit `assert`, dass beide Dicts gleich sind.

Zusatzaufgabe: Öffnen Sie `out/title_counts.json` in der Zelle nach der Prüfzelle noch einmal zum Lesen und geben Sie die ersten fünf Zeilen unverändert aus, dazu die erste Zeile mit einem Umlaut. Woran erkennen Sie in der Datei die Wirkung von `indent=2` und von `ensure_ascii=False`?

Hinweis: Das Zählmuster lautet `title_counts[title] = title_counts.get(title, 0) + 1` in einer Schleife über `passages`. Die Schlüssel sind Strings, der Rückweg aus JSON liefert deshalb dasselbe Dict.

→ Lösung im Anhang

```
title_counts = {}
# TODO: Ihre Lösung hier
...
```

```
# Prüfzelle – einfach ausführen
assert len(title_counts) == 217, "217 verschiedene Titel"
assert title_counts["Mexiko-Stadt"] == 6, "der Artikel Mexiko-Stadt hat sechs Absätze"
assert sum(title_counts.values()) == 400, "alle 400 Absätze gezählt"
assert Path("out/title_counts.json").exists(), "die Datei out/title_counts.json fehlt"
with open("out/title_counts.json", encoding="utf-8") as _f:
    _text = _f.read()
assert "\n" in _text and "Aufklärung" in _text, "mit indent=2 und ensure_ascii=False schreiben"
assert restored_counts == title_counts, "restored_counts muss dem Original gleichen"
print("Alle Prüfungen bestanden ✓")
```

```
# Zusatzaufgabe: die ersten fünf Zeilen der Datei und eine Zeile mit Umlaut
# TODO: Ihre Lösung hier
...
```

Binärdateien. Nicht jede Datei ist Text. Eine Textdatei besteht aus Bytes, die über die Kodierung Zeichen bedeuten, deshalb liefert `read()` einen String. Eine Binärdatei besteht aus Bytes, deren Bedeutung das Dateiformat festlegt, etwa ein Kopf mit Form und Datentyp und danach die Zahlen selbst, so wie sie im Speicher liegen. Mit `open(path, "rb")` liest `read()` solche Bytes roh; das hilft nur, wenn man das Format selbst auswertet. Üblich ist, die Bibliothek zu verwenden, die das Format kennt. Die Vektoren des Kurses liegen als `.npy`-Dateien vor, dem Format von NumPy für ein einzelnes Array samt Form und Datentyp, und `np.load` liest sie.

```
import numpy as np

E = np.load("data/appetizer_embeddings.npy")          # die 400 Absatzvektoren
Q = np.load("data/appetizer_query_embeddings.npy")    # die 8 Anfragevektoren
print(E.shape, E.dtype, Q.shape, Q.dtype)
```

```
(400, 384) float32 (8, 384) float32
```

`E` und `Q` brauchen Sie in Teil 4 wieder. Mit `np.save("out/x.npy", x)` schreiben Sie ein Array in eine solche Datei. Für Tabellen mit Spalten ist das Format CSV üblich, das in der nächsten Lerneinheit mit pandas gelesen wird.

Lernfragen 3.1 #3

1. Welche Aussagen über `with open(...) as f:` stimmen?
   - Nach dem Block ist die Datei geschlossen, auch wenn im Block ein Fehler aufgetreten ist.
   - `f.write("Zeile")` hängt automatisch einen Zeilenumbruch an.
   - Der Modus `"w"` löscht einen vorhandenen Inhalt der Datei.
   - `json.load` liefert für eine Datei mit `[1, 2]` ein Tupel.
2. Ein Dict `{1: "a", 2: "b"}` wird mit `json.dump` gespeichert und mit `json.load` gelesen. Was liefert danach der Zugriff mit dem Schlüssel `1`, und warum?

→ Lösung im Anhang

Zum Nachschlagen: Python-Tutorial 7.2, Lesen und Schreiben von Dateien und die Dokumentation zu `json` und `pathlib`.

3.2 Fehler und Exceptions

In Teil 1 haben Sie den kleinsten Debugging-Zyklus kennengelernt: Fehlertyp und Meldung lesen, die eigene Zeile finden, korrigieren. Dort war der Fehler in derselben Zelle. In einem Programm aus mehreren Funktionen kann er tief innen entstehen, und die Meldung zeigt den ganzen Weg dorthin. Dieses Kapitel zeigt, wie Sie diesen Weg lesen, wie Sie Fehler gezielt abfangen und wie Ihre eigenen Funktionen Fehler so melden, dass der Aufrufer etwas damit anfangen kann. Die Ladefunktion aus dem letzten Kapitel ist dafür das Beispiel. Sie kann an drei Stellen scheitern:

- die Datei fehlt,
- der Inhalt ist kein gültiges JSON,
- oder der Schlüssel `passages` fehlt.

Das Kapitel verwendet Ihre Funktion `load_passages` aus dem Kapitel 3.1. Die erste Vorgabe-Zelle enthält dieselbe Funktion noch einmal, damit die Demos auch laufen, wenn Sie jenes Kapitel übersprungen haben; die zweite legt zwei absichtlich kaputte Dateien in `out/` an. Führen Sie beide Zellen einfach aus.

```python
# Vorgabe: load_passages wie in der Musterlösung von Aufgabe 3.1 #1 — einfach ausführen
import json
from pathlib import Path

def load_passages(path):
    """Liest die Liste der Absätze aus einer JSON-Datei im Format der Kursdaten."""
    with open(path, encoding="utf-8") as f:
        return json.load(f)["passages"]

Path("out").mkdir(exist_ok=True)
```

```python
# Vorgabe: zwei kaputte Dateien zum Üben — einfach ausführen
with open("out/broken.json", "w", encoding="utf-8") as f:
    f.write('{"passages": [}')                  # kein gültiges JSON: die Klammern passen nicht
with open("out/no_key.json", "w", encoding="utf-8") as f:
    json.dump({"items": []}, f)                 # gültiges JSON, aber der Schlüssel passages fehlt
print(Path("out/broken.json").exists(), Path("out/no_key.json").exists())
```

```
True True
```

Einen Traceback lesen. Die nächste Zelle ruft `load_passages` nicht direkt auf, sondern über zwei weitere Funktionen, und übergibt einen Pfad, den es nicht gibt. Die Zelle endet mit einer Fehlermeldung, das ist beabsichtigt. Die Meldung heißt Traceback, weil sie den Weg der Aufrufe zurückverfolgt.

```python
def first_title(path):
    return load_passages(path)[0]["title"]

def report(path):
    print("Erster Titel:", first_title(path))

report("data/missing.json")
```

```
---------------------------------------------------------------------------
FileNotFoundError                         Traceback (most recent call last)
Cell In[15], line 7
      4 def report(path):
      5     print("Erster Titel:", first_title(path))
----> 7 report("data/missing.json")

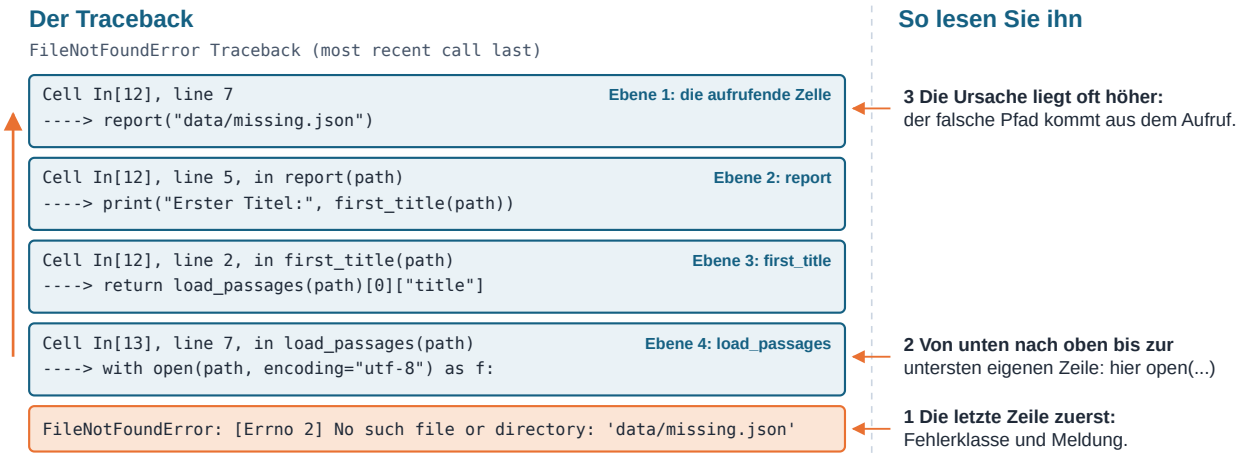
Cell In[15], line 5, in report(path)
      4 def report(path):
----> 5     print("Erster Titel:", first_title(path))

Cell In[15], line 2, in first_title(path)
      1 def first_title(path):
----> 2     return load_passages(path)[0]["title"]

Cell In[13], line 7, in load_passages(path)
      5 def load_passages(path):
      6     """Liest die Liste der Absätze aus einer JSON-Datei im Format der Kursdaten."""
----> 7     with open(path, encoding="utf-8") as f:
      8         return json.load(f)["passages"]

FileNotFoundError: [Errno 2] No such file or directory: 'data/missing.json'
```

So sieht die Meldung im Notebook aus; die Zellnummern und Einzelheiten der Darstellung hängen von der Umgebung ab. Die Grafik zeigt, wie sie zu lesen ist.



Genau verstehen: Die Leserichtung

Ein Traceback besteht aus Ebenen, eine je Funktionsaufruf, in der Reihenfolge der Aufrufe von oben nach unten. Die unterste Ebene ist die Stelle, an der der Fehler entstanden ist. Lesen Sie ihn in drei Schritten:

1. **Die letzte Zeile zuerst.** Sie nennt die Fehlerklasse und die Meldung: `FileNotFoundError: [Errno 2] No such file or directory: 'data/missing.json'`. Damit wissen Sie, was schiefgegangen ist, und oft schon, warum.
2. **Von unten nach oben zur eigenen Zeile.** Die unterste Ebene zeigt die Zeile, an der die Ausnahme ausgelöst wurde, hier `open(...)` in `load_passages`. Gehört sie zu einer Bibliothek, gehen Sie weiter nach oben, bis Sie die unterste Zeile erreichen, die zu Ihrem eigenen Code gehört. Dort setzen Sie an.
3. **Die Ursache liegt oft höher.** In Teil 1 haben Sie gesehen, dass die gemeldete Zeile nicht die Ursachen-Zeile sein muss. Hier ist `open` nicht falsch; der falsche Pfad kommt aus dem Aufruf `report("data/missing.json")` ganz oben. Die Ebenen darüber zeigen, wie der Wert dorthin gelangt ist.

Jede Ebene nennt die Zelle oder Datei, die Zeilennummer und die Funktion, der Pfeil `---->` markiert die Zeile. Bei Fehlern in Bibliotheksfunktionen kommen Ebenen aus Dateien wie `json/decoder.py` dazu, die nächste Demo zeigt das.

```
load_passages("out/broken.json")
```

```
---------------------------------------------------------------------------
JSONDecodeError                           Traceback (most recent call last)
Cell In[16], line 1
----> 1 load_passages("out/broken.json")

Cell In[13], line 8, in load_passages(path)
      6     """Liest die Liste der Absätze aus einer JSON-Datei im Format der Kursdaten."""
      7     with open(path, encoding="utf-8") as f:
----> 8         return json.load(f)["passages"]

File .../json/__init__.py:293, in load(fp, cls, object_hook, parse_float, parse_int, parse_constant,
object_pairs_hook, **kw)
    (gekürzt: weitere Ebenen aus dem Modul json)

File .../json/decoder.py:361, in JSONDecoder.raw_decode(self, s, idx)
    359     obj, end = self.scan_once(s, idx)
    360 except StopIteration as err:
--> 361     raise JSONDecodeError("Expecting value", s, err.value) from None
    362 return obj, end

JSONDecodeError: Expecting value: line 1 column 15 (char 14)
```

Die unterste Ebene liegt in `json/decoder.py`, das ist Bibliothekscode und nicht die Stelle, an der Sie etwas ändern. Die unterste eigene Zeile ist `json.load(f)` in `load_passages`. Die Meldung sagt, dass an Zeichen 14 der Datei ein Wert erwartet wurde, und dort steht die schließende Klammer. Der Fehler liegt also in der Datei, nicht im Code.

Abfangen mit `try` und `except`. Bisher hat jede Ausnahme die Zelle beendet: Python hat die Ausführung abgebrochen und den Traceback ausgegeben. Mit `try` und `except` behandeln Sie eine Ausnahme stattdessen selbst. Der `try`-Block enthält den Code, der scheitern kann; der `except`-Block legt fest, was in diesem Fall geschehen soll, etwa eine Meldung ausgeben und mit einem Ersatzwert weitermachen. Die nächste Zelle lädt die Daten über eine Funktion, die die beiden Fehlerfälle meldet und dann mit einer leeren Liste weitermacht.

```
  def try_load(path):
①     try:
          passages = load_passages(path)
②     except FileNotFoundError as e:
          print("Datei fehlt:", e)
          return []
③     except ValueError as e:
          print("Inhalt ungültig:", e)
          return []
      print(len(passages), "Absätze aus", path)
      return passages

  try_load("data/appetizer_passages.json")
  try_load("data/missing.json")
  result = try_load("out/broken.json")
  print(result)                                  # die leere Liste nach dem abgefangenen Fehler
```

① der `try`-Block enthält den Code, der scheitern kann
② `except FileNotFoundError as e` läuft nur, wenn im try-Block genau diese Ausnahme (oder eine Unterklasse) entsteht; `e` ist das Ausnahmeobjekt, `str(e)` seine Meldung
③ mehrere `except`-Blöcke nacheinander: Python nimmt den ersten, der zur Ausnahme passt; `ValueError` fängt auch den `JSONDecodeError`

```
400 Absätze aus data/appetizer_passages.json
Datei fehlt: [Errno 2] No such file or directory: 'data/missing.json'
Inhalt ungültig: Expecting value: line 1 column 15 (char 14)
[]
```

Der Ablauf ist immer derselbe. Python führt den `try`-Block aus. Entsteht darin eine Ausnahme, wird der Rest des Blocks übersprungen, und Python sucht den ersten `except`-Block, dessen Klasse zur Ausnahme passt. Danach geht es nach dem ganzen `try`-Konstrukt weiter. Passt kein `except`, wandert die Ausnahme zum Aufrufer, und so weiter, bis sie jemand abfängt oder die Zelle endet. Dadurch entsteht ein Traceback wie im Beispiel oben. Entsteht keine Ausnahme, werden die `except`-Blöcke übersprungen.

Zwei Schreibweisen ergänzen das Muster: `except (FileNotFoundError, ValueError):` fängt mehrere Klassen mit demselben Block, und `except ValueError:` ohne `as e` genügt, wenn die Meldung nicht gebraucht wird. In C++ entspricht `except` dem `catch`, und `raise` weiter unten dem `throw`.

`else` und `finally`. Zwei weitere Blöcke können an ein `try` anschließen. Ein `else`-Block läuft nur, wenn im `try`-Block keine Ausnahme aufgetreten ist; dorthin gehört der Code, der das Ergebnis weiterverwendet, damit nicht auch seine Fehler vom `except` abgefangen werden. Ein `finally`-Block läuft immer, mit und ohne Ausnahme, auch wenn ein `except` die Funktion per `return` verlässt; er ist für Aufräumarbeiten gedacht, etwa eine Verbindung zu schließen. Für Dateien übernimmt das schon der `with`-Block.

```
    def try_load_report(path):
        try:
            passages = load_passages(path)
        except FileNotFoundError as e:
            print("Datei fehlt:", e)
①       else:
            print(len(passages), "Absätze aus", path)
②       finally:
            print("Versuch beendet:", path)

    try_load_report("data/appetizer_passages.json")
    try_load_report("data/missing.json")
```

① `else` läuft nur, wenn der `try`-Block ohne Ausnahme durchgelaufen ist
② `finally` läuft in jedem Fall, nach dem `try`-Block, nach einem `except` oder nach `else`

```
400 Absätze aus data/appetizer_passages.json
Versuch beendet: data/appetizer_passages.json
Datei fehlt: [Errno 2] No such file or directory: 'data/missing.json'
Versuch beendet: data/missing.json
```

Fehler auslösen mit `raise`. Eine Funktion kann selbst eine Ausnahme auslösen, wenn sie mit ihrer Eingabe nichts anfangen kann. Das haben Sie im Termin zu LE03 bei `check_input` gesehen.

> 🔄 **Wiederholung: check_input aus LE03**
>
> Die Eingabeprüfung des Operators `row_l2_normalize` (LE 3, Abschnitt 3.5.1) prüft jeden Punkt des Vertrags und bricht mit `raise ValueError(...)` ab, wenn einer verletzt ist, zum Beispiel `if x.ndim != 2: raise ValueError(f"input must have two axes, got {x.ndim}")`. Der Aufrufer bekommt so eine Meldung, die den Grund nennt, statt einer falschen Zahl ohne Warnung.

```
    def score_label(score):
①       if not 0 <= score <= 1:
            raise ValueError(f"score out of range: {score}")
        if score >= 0.7:
            return "gut"
        return "schwach"

    print(score_label(0.89))
    try:
        print(score_label(1.5))
    except ValueError as e:
        print("Abgefangen:", e)
```

① `raise` erzeugt ein Ausnahmeobjekt mit einer Meldung und bricht die Funktion sofort ab; der Aufrufer kann die Ausnahme abfangen oder bekommt den Traceback

```
gut
Abgefangen: score out of range: 1.5
```

Fehlerklassen. `FileNotFoundError` und `JSONDecodeError` sind Klassen, und sie sind in einer Hierarchie geordnet. Alle hier behandelten Fehlerklassen sind Unterklassen von `Exception`. `JSONDecodeError` ist eine Unterklasse von `ValueError`, denn ein ungültiger Inhalt ist ein ungültiger Wert. `FileNotFoundError` gehört zu `OSError`, den Fehlern des Betriebssystems. `KeyError` und `IndexError` sind beide `LookupError`, ein Zugriff mit einem Schlüssel oder Index, den es nicht gibt. Die Hierarchie ist beim Abfangen wichtig: Ein `except ValueError` fängt auch jeden `JSONDecodeError` ab.

Überblick: Die häufigsten Fehlerklassen

Alle sind Unterklassen von `Exception`. Der Pfeil ↳ markiert eine Unterklasse der Klasse darüber.

Klasse	Auslöser	Beispiel
`NameError`	ein Name ist nicht definiert	Tippfehler im Namen, Zelle nicht ausgeführt
↳ `UnboundLocalError`	ein lokaler Name wird gelesen, bevor er einen Wert hat	`count += 1` in einer Funktion, die `count` weder als Parameter bekommt noch vorher setzt
`TypeError`	ein Wert hat den falschen Typ, oder ein Aufruf passt nicht zur Signatur	`"3" + 1`, `len(5)`, unbekanntes Argument
`ValueError`	der Typ passt, der Wert nicht	`int("x")`, `float("")`
↳ `json.JSONDecodeError`	Text ist kein gültiges JSON	`json.load` auf eine kaputte Datei
`LookupError`	ein Nachschlagen in einem Container scheitert	die beiden folgenden
↳ `KeyError`	ein Schlüssel fehlt	`d["author"]`
↳ `IndexError`	eine Position ist außerhalb	`xs[100]`
`AttributeError`	ein Attribut oder eine Methode fehlt	`xs.push(1)`, vergessenes `self.`
`OSError`	das Betriebssystem meldet einen Fehler, etwa beim Dateizugriff	die folgende
↳ `FileNotFoundError`	eine Datei oder ein Pfad fehlt	`open("fehlt.json")`
`ZeroDivisionError`	Division durch null	`x / 0`
`AssertionError`	ein `assert` schlägt fehl	eine Prüfzelle meldet einen Fehler

Die Fehlerklasse wählen Sie nach diesem Überblick: `ValueError` für einen unzulässigen Wert, `TypeError` für einen falschen Typ, `KeyError` oder `FileNotFoundError`, wenn Ihre Funktion dieselbe Situation meldet wie die eingebauten. Die Meldung nennt den erwarteten und den tatsächlichen Wert, denn der Aufrufer kennt den Code Ihrer Funktion nicht und muss allein aus der Meldung verstehen, was falsch war. Die Meldung schreiben wir wie den übrigen Code auf Englisch: Sie steht im Traceback neben Pythons eigenen Meldungen, und die sind englisch.

`assert` oder `raise`? Beide brechen ab, sie haben aber verschiedene Aufgaben. `assert` prüft eine Annahme, die im Programm gelten muss, und ist für Tests gedacht, so wie in den Prüfzellen dieses Kurses. Python kann alle `assert`-Anweisungen abschalten, wenn es mit der Option `-O` läuft, deshalb darf eine Funktion ihre Eingaben nicht mit `assert` prüfen. Eingaben prüft die Funktion selbst mit `if` und löst bei einer verletzten Bedingung mit `raise` eine Ausnahme aus. `check_input` ist ein Beispiel hierfür.

Eigene Fehlerklassen. Wenn die eingebauten Klassen nicht treffen oder ein Programm seine eigenen Fehlerarten unterscheiden will, definiert es eine Unterklasse von `Exception`. Der Rumpf braucht nur einen Docstring, geerbt wird alles. Aufrufer können diese Klasse gezielt abfangen, ohne andere Fehler mitzufangen. Wer sie von `ValueError` statt von `Exception` ableitet, erreicht, dass auch ein allgemeines `except ValueError` sie noch fängt. Im Grundkurs reichen die eingebauten Klassen; der Fehlervertrag unten verwendet `ValueError`.

```
① class CorpusError(Exception):
      """Die Kursdaten haben nicht das erwartete Format."""

  def passages_of(data, path):
      if "passages" not in data:
②         raise CorpusError(f"key 'passages' missing in {path}")
      return data["passages"]

  try:
      passages_of({"items": []}, "out/no_key.json")
③ except CorpusError as e:
      print("Kursdaten unbrauchbar:", e)
```

① eine eigene Fehlerklasse: Unterklasse von `Exception`, der Rumpf ist nur ein Docstring
② sie wird wie jede andere ausgelöst, mit einer Meldung
③ und genauso gezielt abgefangen

```
Kursdaten unbrauchbar: key 'passages' missing in out/no_key.json
```

Überblick: Abfragen oder abfangen?

Wenn ein Zugriff scheitern kann, gibt es zwei Wege: vorher prüfen (`if key in d`, `path.exists()`) oder den Fehler abfangen (`try` / `except`). Welcher Weg passt, hängt vom Fall ab.

	Abfragen	Abfangen
Beispiel	`if "passages" in data:` oder `d.get(key, default)`	`try: open(path) … except FileNotFoundError:`
Passt, wenn	die Prüfung wenig kostet (ein `in` auf einem Dict) und jeden Fehlerfall sicher erkennt, nach der Prüfung also nichts mehr schiefgehen kann	der Zugriff selbst den Fehler schon meldet, oder sich zwischen Prüfung und Zugriff etwas ändern kann, etwa weil ein anderes Programm die Datei zwischendurch löscht
Beispiel aus diesem Teil	Schlüssel in einem Dict: `in` ist klar und billig, `get` noch kürzer	Dateien: Zwischen `exists()` und `open` kann die Datei verschwinden, und `open` meldet den Fehler von selbst mit dem Pfad

Eine Regel „immer abfangen" gibt es nicht. Eine Prüfung mit `in` liest sich oft leichter als ein `try`-Block.

Fangen Sie nur die Klassen ab, mit denen Sie an der Stelle etwas anfangen können; der Review am Ende dieses Kapitels zeigt, was ein zu breites `except` anrichtet.

Der dritte Fehlerfall der Ladefunktion ist ein fehlender Schlüssel. Die nächste Zelle zeigt seinen Traceback, er ist kurz.

```
  load_passages("out/no_key.json")
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[21], line 1
----> 1 load_passages("out/no_key.json")

Cell In[13], line 8, in load_passages(path)
      6     """Liest die Liste der Absätze aus einer JSON-Datei im Format der Kursdaten."""
      7     with open(path, encoding="utf-8") as f:
----> 8         return json.load(f)["passages"]

KeyError: 'passages'
```

✏️ Aufgabe 3.2 #1 10 min

In der Zelle unten stehen drei Aufrufe von `load_passages`. Jeder löst beim ersten Ausführen einen anderen Fehler aus.

1. Führen Sie die Zelle aus. Sie bricht beim ersten Aufruf ab. Lesen Sie den Traceback wie oben beschrieben und notieren Sie als Kommentar über dem Aufruf: die Fehlerklasse, die Ursache und die unterste Zeile, die zu Ihrem eigenen Code gehört.
2. Beheben Sie die Ursache. Beim ersten Aufruf liegt sie im Pfad. Bei den beiden anderen liegt sie in der Datei: Schreiben Sie die Datei vor dem Aufruf neu, mit gültigem Inhalt: `with open("out/broken.json", "w", encoding="utf-8") as f:` und darunter `json.dump({"passages": []}, f)`, für die dritte Datei entsprechend.
3. Führen Sie die Zelle nach jeder Korrektur erneut aus, bis alle drei Aufrufe durchlaufen und die letzte Zeile `400 0 0` ausgibt.

Hinweis: Beim zweiten Aufruf zeigt die unterste Ebene in das Modul `json`. Gehen Sie nach oben bis zur Zeile mit `json.load` in `load_passages`: Der Code ist richtig, der Inhalt der Datei nicht.

→ Lösung im Anhang

```
first = load_passages("appetizer_passages.json")
second = load_passages("out/broken.json")
third = load_passages("out/no_key.json")
print(len(first), len(second), len(third))
```

```
# Prüfzelle – einfach ausführen
assert len(first) == 400, "der erste Aufruf lädt die Kursdaten aus data/"
assert second == [] and third == [], "die beiden Dateien in out/ enthalten jetzt eine leere Liste unter passages"
with open("out/broken.json", encoding="utf-8") as _f:
    assert "passages" in json.load(_f), "out/broken.json ist wieder gültiges JSON mit dem Schlüssel passages"
with open("out/no_key.json", encoding="utf-8") as _f:
    assert "passages" in json.load(_f), "out/no_key.json hat jetzt den Schlüssel passages"
print("Alle Prüfungen bestanden ✓")
```

Ein Fehlervertrag für die Ladefunktion. Der Aufrufer von `load_passages` muss wissen, welche Fehler sie melden kann, sonst lässt sich kein passendes `except` schreiben. Für die drei Fälle legen wir fest:

1. **Die Datei fehlt:** `FileNotFoundError`, so wie `open` ihn auslöst. Seine Meldung nennt den Pfad, die Funktion reicht ihn unverändert weiter.
2. **Der Inhalt ist kein gültiges JSON:** `ValueError`, so wie `json.load` ihn auslöst (als `JSONDecodeError`). Auch er wird unverändert weitergereicht.
3. **Der Schlüssel `passages` fehlt:** `ValueError` mit einer eigenen Meldung, die den Schlüssel und den Pfad nennt. Ohne diese Prüfung meldet die Funktion einen `KeyError: 'passages'`, der nicht sagt, welche Datei betroffen ist.

Für die ersten beiden Fälle ist nichts zu tun, die eingebauten Fehler sind schon verständlich. Der dritte Fall ist ein Fall für Abfragen: `"passages" in data` ist billig und vollständig.

Der Vertrag prüft die JSON-Syntax und den Schlüssel, mehr nicht. `{"passages": 42}` ist gültiges JSON mit dem Schlüssel, und die Funktion gibt dann `42` zurück. Dass der Wert eine Liste von Absätzen mit den erwarteten Schlüsseln ist, setzen wir voraus, so wie die Prüfzellen in LE03 die Kursdaten voraussetzen. Die Zusatzaufgabe zieht die Grenze einen Schritt weiter.

Eine gute Stelle, den Vertrag festzuhalten, ist der Docstring der Funktion, mit einem Satz je Fehlerfall; wer `help(load_passages)` aufruft, sieht ihn dann. Die Musterlösung und die Fassung in `tools.py` im nächsten Kapitel machen es so.

✏️ Aufgabe 3.2 #2 10 min

Schreiben Sie `load_passages_checked(path)` nach dem Fehlervertrag oben. Die Funktion liest die Datei wie `load_passages`, prüft aber nach dem Laden, ob der Schlüssel `"passages"` vorhanden ist, und löst sonst einen `ValueError` aus, dessen Meldung den Schlüssel und den Pfad nennt. Im Normalfall gibt sie die Liste zurück. Die Prüfzelle legt eigene Dateien in `out/` an und prüft alle drei Fehlerfälle mit `try` und `except`, so wie die Prüfzellen in LE03.

Zusatzaufgabe: Schreiben Sie in der Zelle nach der Prüfzelle eine Fassung `load_passages_strict(path)`, die zusätzlich prüft, dass der Wert unter `"passages"` eine Liste ist, und sonst einen `ValueError` mit Pfad auslöst. `isinstance(x, list)` liefert `True`, wenn `x` eine Liste ist. Probieren Sie die Funktion an einer Datei mit dem Inhalt `{"passages": 42}` aus, mit `try` und `except ValueError as e`.

Hinweis: Laden Sie den Inhalt zuerst in eine Variable `data`. Dann `if "passages" not in data: raise ValueError(f"...")`, und erst danach `return data["passages"]`.

→ Lösung im Anhang

```python
def load_passages_checked(path):
    # TODO: Ihre Lösung hier
    ...
```

```python
# Prüfzelle – einfach ausführen (legt eigene Dateien in out/ an)
with open("out/no_key_2.json", "w", encoding="utf-8") as _f:
    json.dump({"items": []}, _f)
with open("out/broken_2.json", "w", encoding="utf-8") as _f:
    _f.write('{"passages": []}')
assert load_passages_checked("data/appetizer_passages.json") is not None, "load_passages_checked gibt noch nichts zurück"
assert len(load_passages_checked("data/appetizer_passages.json")) == 400, "im Normalfall die Liste der Absätze"
with open("out/two.json", "w", encoding="utf-8") as _f:
    json.dump({"passages": [{"id": 1, "title": "A", "text": "a"}, {"id": 2, "title": "B", "text": "b"}]}, _f)
assert len(load_passages_checked("out/two.json")) == 2, "die Funktion liest die Datei unter path, nicht die globale Liste passages"
try:
    load_passages_checked("out/no_key_2.json")
    raise AssertionError("eine Datei ohne passages muss einen ValueError auslösen")
except (ValueError, KeyError) as e:
    assert type(e) != KeyError, "ein KeyError ist nicht der vereinbarte Fehler: vorher mit in prüfen und einen ValueError auslösen"
    assert "passages" in str(e) and "no_key_2.json" in str(e), "die Meldung nennt den Schlüssel und den Pfad"
try:
    load_passages_checked("out/broken_2.json")
    raise AssertionError("ungültiges JSON muss einen ValueError auslösen, json.load tut das von selbst")
except json.JSONDecodeError as e:
    assert "line 1 column 15" in str(e), "den JSONDecodeError von json.load unverändert weiterreichen, nicht ersetzen"
except ValueError:
    raise AssertionError("der JSONDecodeError von json.load soll unverändert weitergereicht werden, nicht durch einen eigenen ValueError ersetzt")
try:
    load_passages_checked("out/gibt_es_nicht.json")
    raise AssertionError("eine fehlende Datei muss einen FileNotFoundError auslösen, open tut das von selbst")
except FileNotFoundError as e:
    assert "gibt_es_nicht.json" in str(e), "die Meldung von open nennt den Pfad"
print("Alle Prüfungen bestanden ✓")
```

```
# Zusatzaufgabe: der Fehlervertrag einen Schritt weiter
# TODO: Ihre Lösung hier
...
```

✏️ Aufgabe 3.2 #3 10 min

Review. Ein Kollege legt `load_passages_safe` in der Zelle darunter vor. Die Anforderung lautet: Die Datei ist optional. Fehlt sie, gibt die Funktion eine leere Liste zurück. Alle anderen Fehler müssen sichtbar bleiben.

1. Führen Sie die Zelle aus und rufen Sie die Funktion in einer neuen Zelle für `"out/gibt_es_nicht.json"` und für `"out/no_key_2.json"` auf. Was kommt jeweils zurück?
2. Bauen Sie zur Probe einen Tippfehler in den Funktionskörper ein, etwa `["pasages"]` statt `["passages"]`, und rufen Sie die Funktion für die Kursdaten auf. Was kommt zurück, und was erfahren Sie über Ihren Tippfehler? Machen Sie den Tippfehler danach rückgängig.
3. Notieren Sie Ihr Urteil als Kommentar und korrigieren Sie die Funktion direkt in der Zelle, sodass sie die Anforderung erfüllt.

Hinweis: Die Frage ist, welche Fehlerklasse die Anforderung „Datei ist optional" tatsächlich meint. Siehe Box „Überblick: Die häufigsten Fehlerklassen".

→ Lösung im Anhang

```
# Die zu prüfende Fassung — korrigieren Sie nur, was wirklich falsch ist
def load_passages_safe(path):
    try:
        with open(path, encoding="utf-8") as f:
            return json.load(f)["passages"]
    except Exception:
        return []
```

```
# Prüfzelle — einfach ausführen (verwendet die Dateien der vorigen Prüfzelle)
assert load_passages_safe("out/gibt_es_nicht.json") == [], "eine fehlende Datei ergibt eine leere Liste"
assert len(load_passages_safe("data/appetizer_passages.json")) == 400
assert len(load_passages_safe("out/two.json")) == 2, "die Funktion liest die Datei unter path"
try:
    load_passages_safe("out")
    raise AssertionError("ein Ordner als Pfad ist keine fehlende Datei: nur FileNotFoundError abfangen,
nicht OSError")
except (IsADirectoryError, PermissionError):
    pass
try:
    load_passages_safe("out/no_key_2.json")
    raise AssertionError("ein fehlender Schlüssel darf nicht verschluckt werden: nur FileNotFoundError
abfangen")
except KeyError:
    pass
try:
    load_passages_safe("out/broken_2.json")
    raise AssertionError("ungültiges JSON darf nicht verschluckt werden")
except ValueError:
    pass
print("Alle Prüfungen bestanden ✓")
```

Ein zu breites `except` ist ein typisches Fehlerbild in KI-generiertem Code: Das Programm läuft scheinbar fehlerfrei und liefert leere oder falsche Ergebnisse. Fangen Sie nur die Klassen ab, die Sie erwarten und behandeln können. Alles andere soll abbrechen, denn ein Traceback ist eine Information, kein Schönheitsfehler. Teil 4 des Grundkurses kommt darauf zurück.

Lernfragen 3.2 #1

1. Welche Aussagen stimmen?
   - `except ValueError` fängt auch einen `json.JSONDecodeError` ab.
   - Nach einer abgefangenen Ausnahme macht Python mit der nächsten Zeile im `try`-Block weiter.
   - Eine Ausnahme, die keine Funktion abfängt, beendet die Zelle mit einem Traceback.
   - `assert` eignet sich, um in einer Funktion die Eingaben zu prüfen.
2. `score_label(1.5)` (die Funktion von oben) bricht mit einem `ValueError` ab. In welcher Zeile steht die Ursache, und in welcher Zeile entsteht die Ausnahme?

→ Lösung im Anhang

Zum Nachschlagen: Python-Tutorial 8. Fehler und Ausnahmen und die Hierarchie der eingebauten Ausnahmen.

3.3 Module und Imports

`import` kennen Sie seit Teil 1. Dieses Kapitel ordnet die Schreibweisen und zeigt, wie Sie eigene Funktionen in ein Modul auslagern, damit mehrere Notebooks dieselben Hilfsfunktionen verwenden können. Ein Modul ist eine Datei mit Python-Code, ihr Name ohne `.py` ist der Name beim Import.

```
① import math
  print(math.sqrt(2))
② from math import sqrt
  print(sqrt(2))
③ import numpy as np
  print(np.sqrt(2))
```

① `import math` macht das Modul unter seinem Namen verfügbar, der Zugriff läuft über die Punktnotation
② `from math import sqrt` holt nur den Namen `sqrt` ins Notebook, ohne Präfix
③ `import numpy as np` vergibt einen Kurznamen, im Kurs die übliche Form für numpy, pandas und matplotlib

```
1.4142135623730951
1.4142135623730951
1.4142135623730951
```

Welche Form passt? `import m` hält die Herkunft sichtbar: Bei `np.sqrt` ist zu sehen, dass NumPy rechnet. `from m import f` passt für einzelne, häufig gebrauchte Namen. `from m import *` holt alle Namen des Moduls ungesehen ins Notebook; dann ist nicht mehr erkennbar, woher ein Name stammt, und gleichnamige Funktionen überschreiben sich. Diese Form kommt im Kurs nicht vor und wird allgemein nicht empfohlen.

Wo Python sucht. Beim `import` geht Python die Liste `sys.path` der Reihe nach durch, eine gewöhnliche Python-Liste von Ordnerpfaden im Modul `sys` (`import sys` und `print(sys.path)` zeigen sie). Im Notebook steht darin das Arbeitsverzeichnis des Kernels, dazu die installierte Umgebung, also der Ort, an dem `uv` oder Colab die Pakete abgelegt haben (siehe das Begleitdokument zur Entwicklungsumgebung). Eine eigene Datei `tools.py` im Arbeitsverzeichnis ist damit importierbar, und `tools.__file__` zeigt, welche Datei geladen wurde. Daraus folgt eine Regel: Eigene Dateien nie `json.py` oder `numpy.py` nennen, sie verdecken sonst die Bibliothek. Findet Python kein Modul mit dem Namen, meldet es einen `ModuleNotFoundError`; gibt es das Modul, aber nicht den Namen in `from m import name`, einen `ImportError`.

Ein eigenes Modul anlegen. Ein eigenes Modul ist nichts anderes als eine `.py`-Datei neben dem Notebook; Sie könnten `tools.py` mit einem Editor anlegen und darin Funktionen definieren. Im Notebook geht es ohne Editor: Der Zellbefehl `%%writefile tools.py` schreibt den Inhalt der Zelle in diese Datei, statt ihn als Python auszuführen. Die Datei landet im Arbeitsverzeichnis des Kernels, und der Befehl überschreibt eine vorhandene Datei vollständig. Die nächste Zelle legt so das Modul mit der Funktion `clean_words` aus Teil 2 an, die Zelle danach importiert es.

```
%%writefile tools.py
"""Hilfsfunktionen für die Suche: Text bereinigen, Kursdaten laden."""
import json

PUNCTUATION = ".,;:!?()\"„“"


def clean_words(text):
    """Gibt die Wörter des Textes in Kleinbuchstaben und ohne Satzzeichen an den Wortenden zurück."""
    result = []
    for word in text.lower().split():
        word = word.strip(PUNCTUATION)
        if word != "":
            result.append(word)
    return result
```

```
import tools

print(tools.clean_words("Die GPU, die gpu!"))
print(tools.PUNCTUATION)
```

```
['die', 'gpu', 'die', 'gpu']
.,;:!?()"„“
```

```
print(tools.__file__)       # welche Datei geladen wurde: bei Ihnen tools.py in Ihrem Notebook-Ordner
```

Zwei Dinge sind dabei wichtig. Erstens hat das Modul seinen eigenen Namensraum: `tools.py` muss `json` selbst importieren, wenn eine seiner Funktionen `json.load` aufruft. Ein `import json` im Notebook gilt nur im Notebook. Deshalb steht die Zeile `import json` oben in der Datei, obwohl `clean_words` sie aktuell noch nicht braucht; die nächste Aufgabe braucht sie dann. Zweitens führt Python die Datei beim ersten `import` aus, von oben nach unten: Jedes `def` legt eine Funktion an, ein `print` auf Modulebene gibt etwas aus. Das Ergebnis bleibt im Speicher, ein erneutes `import tools` führt nichts mehr aus. `import` ist also keine Textersetzung wie `#include` in C. Ändern Sie `tools.py`, laden Sie das Modul mit `importlib.reload(tools)` neu, oder Sie starten den Kernel neu. Die nächsten beiden Zellen zeigen das an einer Minidatei.

```
%%writefile demo.py
print("demo.py wird ausgeführt, __name__ ist", __name__)

def f():
    return 42

if __name__ == "__main__":
    print("direkt gestartet:", f())
```

```
import demo          # erster Import: die Datei läuft, die Ausgabe erscheint
import demo          # zweiter Import: nichts, das Modul ist schon im Speicher
print(demo.f())
```

```
demo.py wird ausgeführt, __name__ ist demo
42
```

Beim Import heißt das Modul wie die Datei, `__name__` ist `"demo"`, und der Block unter `if __name__ == "__main__":` wird übersprungen. Starten Sie die Datei dagegen direkt, mit `python demo.py` im Terminal, ist `__name__` gleich `"__main__"`, und der geschützte Block läuft mit. So trennt die Zeile am Ende vieler Python-Dateien einen Testaufruf, der nur beim direkten Start laufen soll, von den Definitionen, die der Import bereitstellt. Ein Paket ist ein Ordner mit Modulen, also mit `.py`-Dateien; `numpy` ist eines, und `np.linalg` ist ein Modul darin.

Noch ein Hinweis zu `reload`: Es erneuert das Modul, nicht die Namen, die Sie vorher mit `from tools import …` ins Notebook geholt haben; die zeigen weiter auf die alte Fassung. Solange Sie an einem Modul arbeiten, greifen Sie deshalb über `tools.` zu oder wiederholen den `from`-Import nach jedem `reload`.

✏️ Aufgabe 3.3 #1 10 min

Erweitern Sie `tools.py` um Ihre Funktion `load_passages_checked` aus dem Kapitel 3.2, im Modul unter dem Namen `load_passages`. Die Zelle darunter enthält die vollständige Datei, denn `%%writefile` überschreibt sie. Ergänzen Sie die Funktion an der markierten Stelle, führen Sie die Zelle aus und laden Sie das Modul in der zweiten Zelle mit `importlib.reload(tools)` neu. Rufen Sie danach `tools.load_passages("data/appetizer_passages.json")` auf.

Zusatzaufgabe: Schreiben Sie eine Funktion `count_titles(passages)`, die das Dict von Titel zu Anzahl liefert, nach dem Zählmuster aus Aufgabe 3.1 #2 im Kapitel 3.1. Nehmen Sie sie in das Modul auf, laden Sie das Modul neu und rufen Sie die Funktion über `tools.` auf.

Hinweis: Die Funktion ist dieselbe wie im Fehlerkapitel, im Modul heißt sie nur `load_passages`. Das `json`, das sie aufruft, kommt aus der Zeile `import json` oben in `tools.py`, nicht aus dem Notebook.

→ Lösung im Anhang

```
%%writefile tools.py
"""Hilfsfunktionen für die Suche: Text bereinigen, Kursdaten laden."""
import json

PUNCTUATION = ".,;:!?()\"„“"

def clean_words(text):
    """Gibt die Wörter des Textes in Kleinbuchstaben und ohne Satzzeichen an den Wortenden zurück."""
    result = []
    for word in text.lower().split():
        word = word.strip(PUNCTUATION)
        if word != "":
            result.append(word)
    return result

def load_passages(path):
    """Liest die Absätze aus einer JSON-Datei im Format der Kursdaten.

    Fehlt die Datei, löst open einen FileNotFoundError aus; ist der Inhalt kein
    gültiges JSON, löst json.load einen ValueError aus. Fehlt der Schlüssel
    passages, löst die Funktion selbst einen ValueError mit Schlüssel und Pfad aus.
    """
    # TODO: Ihre Lösung hier
    ...
```

```
import importlib
# TODO: Ihre Lösung hier
...
```

```
# Prüfzelle – einfach ausführen (lädt das Modul selbst neu und legt ihre Datei in out/ an)
import importlib
importlib.reload(tools)
with open("out/no_key_tools.json", "w", encoding="utf-8") as _f:
    _f.write('{"items": []}')
① assert hasattr(tools, "load_passages"), "tools.py enthält noch keine Funktion load_passages"
② assert tools.load_passages.__module__ == "tools", "load_passages muss in tools.py stehen, nicht im Notebook"
assert len(tools.load_passages("data/appetizer_passages.json")) == 400, "die Funktion liest die Kursdaten"
assert tools.clean_words("Die GPU!") == ["die", "gpu"], "clean_words muss in der Datei bleiben"
try:
    tools.load_passages("out/no_key_tools.json")
    raise AssertionError("eine Datei ohne passages muss einen ValueError auslösen")
except ValueError as e:
    assert "passages" in str(e), "die Meldung nennt den Schlüssel"
print("Alle Prüfungen bestanden ✓")
```

① `hasattr(obj, "name")` prüft, ob ein Objekt (hier das Modul) ein Attribut dieses Namens hat
② `f.__module__` nennt das Modul, in dem eine Funktion definiert wurde

```
# Zusatzaufgabe: count_titles schreiben und ins Modul aufnehmen
# TODO: Ihre Lösung hier
...
```

In der Projektaufgabe der nächsten Lerneinheit liegen Hilfsfunktionen wie diese in einer Datei neben dem Notebook, und Teil 4 des Grundkurses verwendet `tools.py` wieder. Sie wissen jetzt, wie die Funktionen dorthin kommen und wie Sie sie laden.

Lernfragen 3.3 #1

1. `tools.py` enthält `def load_passages(path): ... json.load(f) ...`, aber keine Zeile `import json`. Das Notebook hat `json` importiert. Was passiert beim Aufruf `tools.load_passages(...)`?
2. Sie haben `tools.py` geändert und führen `import tools` erneut aus. Warum läuft weiter die alte Fassung, und was hilft?

→ Lösung im Anhang

Zum Nachschlagen: Python-Tutorial 6, Module, die Dokumentation zu `importlib.reload` und zum Zellbefehl `%%writefile`.

3.4 Selbsttest

Drei Aufgaben, zusammen 15 Minuten, ohne KI. Die Prüfzellen nennen bei einem Fehler das Kapitel, in dem der Stoff steht. Die ersten beiden Aufgaben verwenden `load_passages_checked` aus dem Kapitel 3.2 und das Modul `tools` aus dem Kapitel 3.3.

✏️ Aufgabe 3.4 #1 5 min

Einen Traceback deuten. Die Zelle darunter enthält drei Funktionen, die einander aufrufen, und bricht mit einem Fehler ab. Führen Sie sie aus, lesen Sie den Traceback von unten nach oben und notieren Sie als Kommentar in der Zelle: die Fehlerklasse, die unterste Zeile Ihres eigenen Codes und die Zeile, in der die Ursache liegt. Korrigieren Sie dann die Ursache, bis `report()` den Titel des ersten Absatzes zurückgibt.

→ Lösung im Anhang

```python
def first_passage(path):
    return load_passages_checked(path)[0]

def first_title(path):
    return first_passage(path)["title"]

def report():
    return first_title("data/apetizer_passages.json")

print(report())
```

```python
# Prüfzelle – einfach ausführen
assert report() == "Aufzugsanlage", "der Fehler liegt im Pfad in report, nicht in der Ladefunktion (Kapitel Fehler und Exceptions)"
print("Alle Prüfungen bestanden ✓")
```

✏️ Aufgabe 3.4 #2 5 min

Ein Modul mit Fehlervertrag verwenden. Importieren Sie in der Zelle darunter die Funktion `load_passages` aus Ihrem Modul `tools` mit `from tools import load_passages`. Rufen Sie sie zweimal mit `try` und `except` auf: einmal mit dem Pfad `"data/gibt_es_nicht.json"`, einmal mit `"out/selbsttest_no_key.json"`, einer Datei ohne den Schlüssel `passages`, die die Zelle zuvor anlegt. Legen Sie die Meldung der jeweiligen Ausnahme als String in `message_missing` und `message_no_key` ab. Welche Fehlerklasse fangen Sie jeweils ab?

→ Lösung im Anhang

```python
with open("out/selbsttest_no_key.json", "w", encoding="utf-8") as f:
    json.dump({"items": []}, f)

# TODO: Ihre Lösung hier
...
```

```python
# Prüfzelle – einfach ausführen
assert "gibt_es_nicht.json" in message_missing, "die Meldung von open nennt den fehlenden Pfad (Kapitel Fehler und Exceptions)"
assert "passages" in message_no_key and "selbsttest_no_key.json" in message_no_key, "die Meldung des Fehlervertrags nennt Schlüssel und Pfad (Kapitel Module und Imports)"
print("Alle Prüfungen bestanden ✓")
```

✏️ Aufgabe 3.4 #3 5 min

Laden mit Fehlervertrag. Schreiben Sie `read_scores(path)`. Die Funktion liest eine Textdatei mit einer Zahl je Zeile und gibt die Zahlen als Liste von Fließkommazahlen zurück. Der Vertrag:

1. Fehlt die Datei, löst `open` wie üblich einen `FileNotFoundError` aus; die Funktion reicht ihn unverändert weiter.
2. Leerzeilen werden übersprungen, Leerraum um eine Zahl ebenfalls (`strip`).
3. Enthält eine Zeile keine Zahl, löst die Funktion einen `ValueError` aus, dessen Meldung die Zeilennummer nennt, ab 1 gezählt, und den Inhalt der Zeile.

Die Prüfzelle legt ihre Dateien in `out/` an.

Hinweis: `float(text)` wandelt einen String in eine Zahl und löst bei `"abc"` selbst einen `ValueError` aus, allerdings ohne Zeilennummer. Fangen Sie ihn in der Schleife ab und lösen Sie Ihren eigenen aus. Die Zeilennummer liefert `enumerate(f, start=1)`.

→ Lösung im Anhang

```python
def read_scores(path):
    # TODO: Ihre Lösung hier
    ...
```

```
# Prüfzelle – einfach ausführen (legt eigene Dateien in out/ an)
with open("out/scores_ok.txt", "w", encoding="utf-8") as _f:
    _f.write("0.5\n\n 0.9 \n0.1\n")
with open("out/scores_bad.txt", "w", encoding="utf-8") as _f:
    _f.write("0.5\n\nabc\n0.1\n")                    # die ungültige Zeile ist die dritte, davor eine
Leerzeile
_scores = read_scores("out/scores_ok.txt")
assert _scores is not None, "read_scores gibt noch nichts zurück"
assert _scores == [0.5, 0.9, 0.1], "Leerzeilen überspringen, Leerraum entfernen (Kapitel Dateien und JSON)"
assert type(_scores[0]) == float, "die Werte sind Fließkommazahlen"
try:
    read_scores("out/scores_bad.txt")
    raise AssertionError("eine Zeile ohne Zahl muss einen ValueError auslösen (Kapitel Fehler und
Exceptions)")
except ValueError as e:
    assert "3" in str(e) and "abc" in str(e), "die Meldung nennt die Zeilennummer 3 (Leerzeilen zählen mit)
und den Inhalt abc"
try:
    read_scores("out/scores_fehlt.txt")
    raise AssertionError("eine fehlende Datei muss einen FileNotFoundError auslösen")
except FileNotFoundError:
    pass
print("Alle Prüfungen bestanden ✓")
```

Die Ausnahme aus `float` wird in `read_scores` abgefangen und durch eine eigene ersetzt, die mehr weiß als `float`: die Zeilennummer. Der Aufrufer bekommt die neue Ausnahme; im Traceback erscheint die ursprüngliche darüber mit dem Vermerk „During handling of the above exception, another exception occurred". Das ist beabsichtigt, beide Meldungen zusammen erklären den Fehler.

Zum Lesen fremden Codes drei Formen des Weiterreichens: Ohne `except` wandert eine Ausnahme von selbst zum Aufrufer. Ein bloßes `raise` in einem `except`-Block löst die gerade behandelte Ausnahme erneut aus, etwa nach einer Protokollausgabe. `raise ValueError(...) from e` ersetzt sie wie in `read_scores`, nennt die Ursache aber ausdrücklich; im Traceback steht dann „The above exception was the direct cause of the following exception".

Vertiefung: Was Python außerdem mitbringt

NICHT PRÜFUNGSRELEVANT

Für zwei Muster dieses Teils bringt Python fertige Werkzeuge mit. Für diesen Kurs brauchen Sie sie nicht; in fremdem Code begegnen sie Ihnen.

`pathlib` vertieft. Ein `Path` liest und schreibt Dateien auch ohne `open`: `read_text` liefert den ganzen Inhalt auf einmal, passt also für alles, was bequem in den Speicher geht, eine feste Grenze gibt es nicht. Außerdem zerlegt ein `Path` Pfade und sucht Dateien:

```
from pathlib import Path
p = Path("out") / "hits.txt"
p.write_text("0.89  Gehirn\n", encoding="utf-8")
p.read_text(encoding="utf-8")       # '0.89  Gehirn\n'
p.name, p.suffix, p.parent          # ('hits.txt', '.txt', PosixPath('out'))
sorted(Path("data").glob("*.json")) # alle JSON-Dateien in data/
```

`csv`. Tabellen mit Trennzeichen liest das Modul `csv` zeilenweise als Listen oder Dicts; in der nächsten Lerneinheit übernimmt das die Bibliothek pandas:

```
import csv
with open("out/hits.csv", encoding="utf-8", newline="") as f:
    for row in csv.reader(f):       # je Zeile eine Liste von Strings
        print(row)
```

➡️ Nächste Schritte

- Behalten Sie `tools.py` und den Ordner `out/`: Die Projektaufgabe der nächsten Lerneinheit baut das Suchwerkzeug von der Datei bis zur geprüften Trefferliste und verwendet Ihre Ladefunktion und `tools.py`.
- Arbeiten Sie danach Teil 4 des Grundkurses durch: Klassen nach dem Muster von PyTorch, Signaturen mit `*args` und `**kwargs`, Typannotationen und ein Stilkapitel.
- Wenn ein Traceback aus einer Bibliothek kommt, lesen Sie ihn wie hier geübt: letzte Zeile, dann von unten nach oben bis zur eigenen Zeile. Zum Nachschlagen: das offizielle Python-Tutorial, Kapitel 6 bis 8.

Quellen und Referenzen

- Python Software Foundation: Python-Tutorial, Kapitel 6 (Module), 7.2 (Dateien) und 8 (Fehler und Ausnahmen); die Bibliotheksdokumentation zu `json`, `pathlib`, `importlib` und zur Hierarchie der Ausnahmen. Die Kapitel dieses Teils nennen ihre Abschnitte direkt.
- Python Software Foundation: PEP 686 zur Zeichenkodierung ab Python 3.15.
- IPython: Dokumentation zum Zellbefehl `%%writefile`.
- E. Matthes: *Python Crash Course*, 3. Auflage, Kapitel 10 (Dateien und Ausnahmen). Vertiefung, Text und Beispiele dieses Teils sind eigene.
- Datengrundlage: GermanQuAD (deepset), Lizenz CC BY 4.0. Die Absatztexte stammen aus der deutschen Wikipedia (CC BY-SA). Attribution auch in `data/appetizer_passages.json`. Die Vektoren stammen aus dem Appetizer der ersten Lerneinheit.

Dieses Material wurde mit Unterstützung von KI-Werkzeugen entwickelt (Text und Code: Claude und ChatGPT). Alle Inhalte sind von Prof. Dr. Stephan Kurpjuweit konzipiert, geprüft und verantwortet.

Anhang: Lösungen

3.1 Dateien und JSON

💭 Kurz überlegen 3.1 #1

Die nächste Zelle ruft `f.read()` zweimal hintereinander im selben `with`-Block auf. Was liefert der zweite Aufruf?

> Einen leeren String `""`. Eine geöffnete Datei hat eine Leseposition, die mit jedem Lesen vorrückt; nach dem ersten `read()` steht sie am Dateiende, und dort gibt es nichts mehr zu lesen. `read()` und `readlines()` liefern also den verbleibenden Inhalt, beim ersten Aufruf den ganzen. Deshalb öffnen die Beispiele oben die Datei für jede Variante neu. `readline()` liest genau eine Zeile: Eine Leerzeile liefert `"\n"`, erst das Dateiende liefert `""`.

💭 Kurz überlegen 3.1 #2

Die nächste Zelle öffnet `out/hits.txt` noch einmal mit dem Modus `"w"` und schreibt eine einzige Zeile. Was steht danach in der Datei, und was gibt die Zelle aus?

> Nur die neue Zeile `0.65 Koran`, die Ausgabe ist `['0.65 Koran\n']`. Der Modus `"w"` legt die Datei neu an und löscht einen vorhandenen Inhalt, bevor geschrieben wird. Um an eine Datei anzuhängen, gibt es den Modus `"a"`.

✏️ Aufgabe 3.1 #1 5 min

Schreiben Sie die Funktion `load_passages(path)`. Sie bekommt den Pfad einer JSON-Datei im Format der Kursdaten als String, liest die Datei mit `with open` und `json.load` und gibt die Liste der Absätze zurück, die in der Datei unter dem Schlüssel `"passages"` steht. Rufen Sie die Funktion danach für `"data/appetizer_passages.json"` auf und legen Sie das Ergebnis in der Variablen `passages` ab. Die Beispielanfragen laden Sie auf dieselbe Weise aus `"data/appetizer_queries.json"`, dort heißt der Schlüssel `"queries"`; legen Sie die Liste in der Variablen `queries` ab.

Zusatzaufgabe: Schreiben Sie in der Zelle nach der Prüfzelle eine Funktion `save_titles(records, path)`, die die Titel einer Liste von Absätzen als Textdatei speichert, ein Titel je Zeile, jeder Titel nur einmal und in der Reihenfolge des ersten Vorkommens. Schreiben Sie die Titel von `passages[:10]` nach `out/titles.txt`, lesen Sie die Datei zurück und zählen Sie die Zeilen.

```python
def load_passages(path):
    with open(path, encoding="utf-8") as f:
        return json.load(f)["passages"]

passages = load_passages("data/appetizer_passages.json")
with open("data/appetizer_queries.json", encoding="utf-8") as f:
    queries = json.load(f)["queries"]
```

```python
# Zusatzaufgabe: die Titel einer Absatzliste als Textdatei speichern und zurücklesen
def save_titles(records, path):
    seen = []                                   # Titel in der Reihenfolge des ersten Vorkommens
    for p in records:
        if p["title"] not in seen:
            seen.append(p["title"])
    with open(path, "w", encoding="utf-8") as f:
        for title in seen:
            f.write(title + "\n")

save_titles(passages[:10], "out/titles.txt")
with open("out/titles.txt", encoding="utf-8") as f:
    print(len(f.readlines()), "Zeilen")
```

↑ zur Aufgabenstellung

✏️ Aufgabe 3.1 #2 10 min

Ein typischer Fall für JSON: Ein Zwischenergebnis wird einmal berechnet und als Datei gespeichert, und beim nächsten Start liest das Programm die Datei, statt neu zu rechnen. Hier ist das Zwischenergebnis die Titelstatistik aus Teil 2.

1. Bauen Sie aus `passages` das Dict `title_counts`, das jedem Titel die Anzahl seiner Absätze zuordnet. Das ist das Zählmuster aus Teil 2 mit `get`.
2. Schreiben Sie das Dict mit `json.dump` nach `out/title_counts.json`, lesbar umgebrochen und mit Umlauten als Zeichen.
3. Lesen Sie die Datei mit `json.load` in die neue Variable `restored_counts` zurück und prüfen Sie mit `assert`, dass beide Dicts gleich sind.

Zusatzaufgabe: Öffnen Sie `out/title_counts.json` in der Zelle nach der Prüfzelle noch einmal zum Lesen und geben Sie die ersten fünf Zeilen unverändert aus, dazu die erste Zeile mit einem Umlaut. Woran erkennen Sie in der Datei die Wirkung von `indent=2` und von `ensure_ascii=False`?

```
title_counts = {}
for p in passages:
    title_counts[p["title"]] = title_counts.get(p["title"], 0) + 1

with open("out/title_counts.json", "w", encoding="utf-8") as f:
    json.dump(title_counts, f, indent=2, ensure_ascii=False)

with open("out/title_counts.json", encoding="utf-8") as f:
    restored_counts = json.load(f)
assert restored_counts == title_counts
```

```
# Zusatzaufgabe: die ersten fünf Zeilen der Datei und eine Zeile mit Umlaut
with open("out/title_counts.json", encoding="utf-8") as f:
    lines = f.readlines()
for line in lines[:5]:
    print(line, end="")                  # end="": der Umbruch steht schon in der Zeile
for line in lines:
    if "ä" in line or "ö" in line or "ü" in line:
        print(line, end="")
        break
# indent=2: jeder Eintrag steht in einer eigenen, um zwei Leerzeichen eingerückten Zeile.
# ensure_ascii=False: Umlaute stehen als Zeichen in der Datei, nicht als \u00e4.
```

↑ zur Aufgabenstellung

Lernfragen 3.1 #3

1. Welche Aussagen über `with open(...) as f:` stimmen?

✓ Nach dem Block ist die Datei geschlossen, auch wenn im Block ein Fehler aufgetreten ist. — Das ist der Zweck des Blocks.

✗ `f.write("Zeile")` hängt automatisch einen Zeilenumbruch an. — `write` schreibt genau den übergebenen String. Der Umbruch `\n` muss dabeistehen.

✓ Der Modus `"w"` löscht einen vorhandenen Inhalt der Datei. — Zum Anhängen gibt es den Modus `"a"`.

✗ `json.load` liefert für eine Datei mit `[1, 2]` ein Tupel. — JSON-Arrays werden Listen. Tupel kennt JSON nicht.

2. Ein Dict `{1: "a", 2: "b"}` wird mit `json.dump` gespeichert und mit `json.load` gelesen. Was liefert danach der Zugriff mit dem Schlüssel `1`, und warum?

> Einen `KeyError`. Die Schlüssel eines JSON-Objekts sind Strings, nach dem Rückweg heißen die Schlüssel `"1"` und `"2"`. Der Zugriff lautet deshalb `restored["1"]`.

↑ zur Aufgabenstellung

3.2 Fehler und Exceptions

✏️ Aufgabe 3.2 #1 10 min

In der Zelle unten stehen drei Aufrufe von `load_passages`. Jeder löst beim ersten Ausführen einen anderen Fehler aus.

1. Führen Sie die Zelle aus. Sie bricht beim ersten Aufruf ab. Lesen Sie den Traceback wie oben beschrieben und notieren Sie als Kommentar über dem Aufruf: die Fehlerklasse, die Ursache und die unterste Zeile, die zu Ihrem eigenen Code gehört.
2. Beheben Sie die Ursache. Beim ersten Aufruf liegt sie im Pfad. Bei den beiden anderen liegt sie in der Datei: Schreiben Sie die Datei vor dem Aufruf neu, mit gültigem Inhalt: `with open("out/broken.json", "w", encoding="utf-8") as f:` und darunter `json.dump({"passages": []}, f)`, für die dritte Datei entsprechend.
3. Führen Sie die Zelle nach jeder Korrektur erneut aus, bis alle drei Aufrufe durchlaufen und die letzte Zeile `400 0 0` ausgibt.

```python
# Aufruf 1: FileNotFoundError, No such file or directory: 'appetizer_passages.json'. Der Pfad ohne data/
# zeigt ins Arbeitsverzeichnis, dort liegt keine solche Datei. Eigene Zeile: open(...) in load_passages.
first = load_passages("data/appetizer_passages.json")

# Aufruf 2: JSONDecodeError (ein ValueError), Expecting value: line 1 column 15. Die Datei enthält kein
# gültiges JSON. Unterste eigene Zeile: json.load(f) in load_passages. Abhilfe: Datei neu schreiben.
with open("out/broken.json", "w", encoding="utf-8") as f:
    json.dump({"passages": []}, f)
second = load_passages("out/broken.json")

# Aufruf 3: KeyError: 'passages'. Die Datei ist gültiges JSON, enthält aber den Schlüssel items statt
# passages. Eigene Zeile: der Zugriff ["passages"] in load_passages. Abhilfe: Datei neu schreiben.
with open("out/no_key.json", "w", encoding="utf-8") as f:
    json.dump({"passages": []}, f)
third = load_passages("out/no_key.json")
print(len(first), len(second), len(third))
```

↑ zur Aufgabenstellung

✏️ Aufgabe 3.2 #2 10 min

Schreiben Sie `load_passages_checked(path)` nach dem Fehlervertrag oben. Die Funktion liest die Datei wie `load_passages`, prüft aber nach dem Laden, ob der Schlüssel `"passages"` vorhanden ist, und löst sonst einen `ValueError` aus, dessen Meldung den Schlüssel und den Pfad nennt. Im Normalfall gibt sie die Liste zurück. Die Prüfzelle legt eigene Dateien in `out/` an und prüft alle drei Fehlerfälle mit `try` und `except`, so wie die Prüfzellen in LE03.

Zusatzaufgabe: Schreiben Sie in der Zelle nach der Prüfzelle eine Fassung `load_passages_strict(path)`, die zusätzlich prüft, dass der Wert unter `"passages"` eine Liste ist, und sonst einen `ValueError` mit Pfad auslöst. `isinstance(x, list)` liefert `True`, wenn `x` eine Liste ist. Probieren Sie die Funktion an einer Datei mit dem Inhalt `{"passages": 42}` aus, mit `try` und `except ValueError as e`.

```python
def load_passages_checked(path):
    """Liest die Absätze aus einer JSON-Datei im Format der Kursdaten.

    Fehlt die Datei, löst open einen FileNotFoundError aus; ist der Inhalt kein
    gültiges JSON, löst json.load einen ValueError aus. Fehlt der Schlüssel
    passages, löst die Funktion selbst einen ValueError mit Schlüssel und Pfad aus.
    """
    with open(path, encoding="utf-8") as f:
        data = json.load(f)
    if "passages" not in data:
        raise ValueError(f"key 'passages' missing in {path}")
    return data["passages"]
```

```python
# Zusatzaufgabe: der Fehlervertrag einen Schritt weiter
def load_passages_strict(path):
    with open(path, encoding="utf-8") as f:
        data = json.load(f)
    if not isinstance(data, dict) or "passages" not in data:
        raise ValueError(f"key 'passages' missing in {path}")
    if not isinstance(data["passages"], list):
        raise ValueError(f"'passages' in {path} is not a list")
    return data["passages"]

with open("out/not_a_list.json", "w", encoding="utf-8") as f:
    f.write('{"passages": 42}')
try:
    load_passages_strict("out/not_a_list.json")
except ValueError as e:
    print("Laden fehlgeschlagen:", e)
```

↑ zur Aufgabenstellung

✏️ Aufgabe 3.2 #3 10 min

Review. Ein Kollege legt `load_passages_safe` in der Zelle darunter vor. Die Anforderung lautet: Die Datei ist optional. Fehlt sie, gibt die Funktion eine leere Liste zurück. Alle anderen Fehler müssen sichtbar bleiben.

1. Führen Sie die Zelle aus und rufen Sie die Funktion in einer neuen Zelle für `"out/gibt_es_nicht.json"` und für `"out/no_key_2.json"` auf. Was kommt jeweils zurück?
2. Bauen Sie zur Probe einen Tippfehler in den Funktionskörper ein, etwa `["pasages"]` statt `["passages"]`, und rufen Sie die Funktion für die Kursdaten auf. Was kommt zurück, und was erfahren Sie über Ihren Tippfehler? Machen Sie den Tippfehler danach rückgängig.
3. Notieren Sie Ihr Urteil als Kommentar und korrigieren Sie die Funktion direkt in der Zelle, sodass sie die Anforderung erfüllt.

```
# Die zu prüfende Fassung — korrigieren Sie nur, was wirklich falsch ist
def load_passages_safe(path):
    try:
        with open(path, encoding="utf-8") as f:
            return json.load(f)["passages"]
    except FileNotFoundError:
        return []
# Urteil: Die Vorlage erfüllt die Anforderung nicht. except Exception fängt alle Unterklassen von
Exception ab, auch den
# KeyError bei einer Datei ohne passages, den ValueError bei ungültigem JSON und sogar den eigenen
# Tippfehler ["pasages"]: Alle liefern stillschweigend eine leere Liste, der Fehler bleibt unsichtbar.
# Gemeint ist nur die fehlende Datei, also FileNotFoundError. Alles andere soll weiter abbrechen.
# except OSError wäre schon zu breit: Es fängt auch fehlende Leserechte und andere Fehler des
Betriebssystems ab.
```

↑ zur Aufgabenstellung

Lernfragen 3.2 #1

1. Welche Aussagen stimmen?

✓ `except ValueError` fängt auch einen `json.JSONDecodeError` ab. — `JSONDecodeError` ist eine Unterklasse von `ValueError`.

✗ Nach einer abgefangenen Ausnahme macht Python mit der nächsten Zeile im `try`-Block weiter. — Der Rest des `try`-Blocks wird übersprungen. Es geht nach dem `except`-Block weiter.

✓ Eine Ausnahme, die keine Funktion abfängt, beendet die Zelle mit einem Traceback. — Sie wandert von Aufruf zu Aufruf nach oben, bis sie jemand abfängt oder die Zelle endet.

✗ `assert` eignet sich, um in einer Funktion die Eingaben zu prüfen. — `assert` ist für Tests gedacht und kann abgeschaltet werden. Eingaben prüft die Funktion mit `if` und löst bei einer verletzten Bedingung mit `raise` eine Ausnahme aus.

2. `score_label(1.5)` (die Funktion von oben) bricht mit einem `ValueError` ab. In welcher Zeile steht die Ursache, und in welcher Zeile entsteht die Ausnahme?

> Die Ausnahme entsteht in der Zeile `raise ValueError(...)` in `score_label`, das ist die unterste Ebene des Tracebacks. Die Ursache ist der Wert `1.5` im Aufruf, also die Zeile darüber im Traceback. Die Funktion arbeitet richtig, sie meldet eine unzulässige Eingabe.

↑ zur Aufgabenstellung

3.3 Module und Imports

✏️ Aufgabe 3.3 #1 10 min

Erweitern Sie `tools.py` um Ihre Funktion `load_passages_checked` aus dem Kapitel 3.2, im Modul unter dem Namen `load_passages`. Die Zelle darunter enthält die vollständige Datei, denn `%%writefile` überschreibt sie. Ergänzen Sie die Funktion an der markierten Stelle, führen Sie die Zelle aus und laden Sie das Modul in der zweiten Zelle mit `importlib.reload(tools)` neu. Rufen Sie danach `tools.load_passages("data/appetizer_passages.json")` auf.

Zusatzaufgabe: Schreiben Sie eine Funktion `count_titles(passages)`, die das Dict von Titel zu Anzahl liefert, nach dem Zählmuster aus Aufgabe 3.1 #2 im Kapitel 3.1. Nehmen Sie sie in das Modul auf, laden Sie das Modul neu und rufen Sie die Funktion über `tools.` auf.

```
%%writefile tools.py
"""Hilfsfunktionen für die Suche: Text bereinigen, Kursdaten laden."""
import json

PUNCTUATION = ".,;:!?()\"„“"

def clean_words(text):
    """Gibt die Wörter des Textes in Kleinbuchstaben und ohne Satzzeichen an den Wortenden zurück."""
    result = []
    for word in text.lower().split():
        word = word.strip(PUNCTUATION)
        if word != "":
            result.append(word)
    return result

def load_passages(path):
    """Liest die Absätze aus einer JSON-Datei im Format der Kursdaten.

    Fehlt die Datei, löst open einen FileNotFoundError aus; ist der Inhalt kein
    gültiges JSON, löst json.load einen ValueError aus. Fehlt der Schlüssel
    passages, löst die Funktion selbst einen ValueError mit Schlüssel und Pfad aus.
    """
    with open(path, encoding="utf-8") as f:
        data = json.load(f)
    if "passages" not in data:
        raise ValueError(f"key 'passages' missing in {path}")
    return data["passages"]
```

```
import importlib
importlib.reload(tools)                    # die neue Fassung der Datei laden
passages = tools.load_passages("data/appetizer_passages.json")
print(len(passages), "Absätze über tools.load_passages")
```

```
# Zusatzaufgabe: count_titles schreiben und ins Modul aufnehmen
# In die Zelle mit %%writefile tools.py aufnehmen, dann reload(tools) und tools.count_titles(passages):
#     def count_titles(passages):
#         counts = {}
#         for p in passages:
#             counts[p["title"]] = counts.get(p["title"], 0) + 1
#         return counts
```

↑ zur Aufgabenstellung

Lernfragen 3.3 #1

1. `tools.py` enthält `def load_passages(path): ... json.load(f) ...`, aber keine Zeile `import json`. Das Notebook hat `json` importiert. Was passiert beim Aufruf `tools.load_passages(...)`?

> Ein `NameError: name 'json' is not defined`. Das Modul hat seinen eigenen Namensraum, der Import im Notebook gilt dort nicht. Die Zeile `import json` gehört in die Datei.

2. Sie haben `tools.py` geändert und führen `import tools` erneut aus. Warum läuft weiter die alte Fassung, und was hilft?

> Python führt ein Modul nur beim ersten Import aus und behält es im Speicher. Ein erneutes `import` liefert das vorhandene Modul. `importlib.reload(tools)` führt die Datei neu aus, ebenso ein Neustart des Kernels. Ein vorher mit `from tools import f` geholter Name zeigt nach dem `reload` noch auf die alte Fassung, bis der `from`-Import wiederholt wird.

↑ zur Aufgabenstellung

3.4 Selbsttest

✏️ Aufgabe 3.4 #1 5 min

Einen Traceback deuten. Die Zelle darunter enthält drei Funktionen, die einander aufrufen, und bricht mit einem Fehler ab. Führen Sie sie aus, lesen Sie den Traceback von unten nach oben und notieren Sie als Kommentar in der Zelle: die Fehlerklasse, die unterste Zeile Ihres eigenen Codes und die Zeile, in der die Ursache liegt. Korrigieren Sie dann die Ursache, bis `report()` den Titel des ersten Absatzes zurückgibt.

```python
def first_passage(path):
    return load_passages_checked(path)[0]

def first_title(path):
    return first_passage(path)["title"]

def report():
    return first_title("data/appetizer_passages.json")
# Fehlerklasse: FileNotFoundError, No such file or directory: 'data/apetizer_passages.json'.
# Unterste eigene Zeile: open(...) in load_passages_checked. Ursache: der Tippfehler im Pfad
# in report, drei Ebenen darüber. Die Ladefunktion arbeitet richtig.

print(report())
```

↑ zur Aufgabenstellung

✏️ Aufgabe 3.4 #2 5 min

Ein Modul mit Fehlervertrag verwenden. Importieren Sie in der Zelle darunter die Funktion `load_passages` aus Ihrem Modul `tools` mit `from tools import load_passages`. Rufen Sie sie zweimal mit `try` und `except` auf: einmal mit dem Pfad `"data/gibt_es_nicht.json"`, einmal mit `"out/selbsttest_no_key.json"`, einer Datei ohne den Schlüssel `passages`, die die Zelle zuvor anlegt. Legen Sie die Meldung der jeweiligen Ausnahme als String in `message_missing` und `message_no_key` ab. Welche Fehlerklasse fangen Sie jeweils ab?

```
with open("out/selbsttest_no_key.json", "w", encoding="utf-8") as f:
    json.dump({"items": []}, f)

from tools import load_passages

try:
    load_passages("data/gibt_es_nicht.json")
except FileNotFoundError as e:                  # die fehlende Datei meldet open
    message_missing = str(e)

try:
    load_passages("out/selbsttest_no_key.json")
except ValueError as e:                         # den fehlenden Schlüssel meldet das Modul selbst
    message_no_key = str(e)

print(message_missing)
print(message_no_key)
```

↑ zur Aufgabenstellung

✏️ Aufgabe 3.4 #3 5 min

Laden mit Fehlervertrag. Schreiben Sie `read_scores(path)`. Die Funktion liest eine Textdatei mit einer Zahl je Zeile und gibt die Zahlen als Liste von Fließkommazahlen zurück. Der Vertrag:

1. Fehlt die Datei, löst `open` wie üblich einen `FileNotFoundError` aus; die Funktion reicht ihn unverändert weiter.
2. Leerzeilen werden übersprungen, Leerraum um eine Zahl ebenfalls (`strip`).
3. Enthält eine Zeile keine Zahl, löst die Funktion einen `ValueError` aus, dessen Meldung die Zeilennummer nennt, ab 1 gezählt, und den Inhalt der Zeile.

Die Prüfzelle legt ihre Dateien in `out/` an.

```
def read_scores(path):
    scores = []
    with open(path, encoding="utf-8") as f:
        for number, line in enumerate(f, start=1):
            text = line.strip()
            if text == "":
                continue                        # Leerzeile überspringen
            try:
                scores.append(float(text))
            except ValueError:
                raise ValueError(f"line {number}: not a number: {text}")
    return scores
```

↑ zur Aufgabenstellung